Assignment-4

UCS420: Cognitive Computing
A Cognitive FAQ System Using Pandas (Nova 2.0) – Solved
Roll Number: 1024170236

Q1. Build Your Personalized Knowledge Base

```
Roll number = 1024170236  ->  last two digits = 3, 6

digit 3  ->  category[3 % 3] = category[0] = "billing"
digit 6  ->  category[6 % 3] = category[0] = "billing"

Both digits map to the "billing" category, so 2 new realistic
billing-related FAQ entries were invented for the personalized rows.
```

```
import pandas as pd

fixed_entries = [
 {"question": "what is the annual fee", "answer": "The annual fee is Rs 500.",
  "keywords": "fee cost price charge", "category": "billing"},
 {"question": "how to reset password", "answer": "Go to Settings > Reset Password.",
  "keywords": "password reset login", "category": "account"},
 {"question": "what are your working hours", "answer": "We are open 9 AM to 5 PM.",
  "keywords": "hours timing open time", "category": "general"},
 {"question": "how can i pay the fee", "answer": "You can pay via UPI, card, or net banking.",
  "keywords": "pay payment upi fee", "category": "billing"},
]

personalized_entries = [
 # digit 3 -> billing
 {"question": "how do i get an invoice for my payment",
  "answer": "You can download your invoice anytime from the Billing section of your account.",
  "keywords": "invoice bill receipt", "category": "billing"},
 # digit 6 -> billing
 {"question": "can i pay my fee in installments",
  "answer": "Yes, EMI/installment options are available for fees above Rs 2000.",
  "keywords": "installment emi payment", "category": "billing"},
]

faq_df = pd.DataFrame(fixed_entries + personalized_entries)
print(faq_df)
```

Output (6-row FAQ DataFrame):

```
#   question                                category   keywords
0   what is the annual fee                  billing    fee cost price charge
1   how to reset password                   account    password reset login
2   what are your working hours             general    hours timing open time
3   how can i pay the fee                   billing    pay payment upi fee
4   how do i get an invoice for my payment  billing    invoice bill receipt
5   can i pay my fee in installments        billing    installment emi payment

(answer column omitted here for width; each row has its own full
 answer text, e.g. row 0 -> "The annual fee is Rs 500.")
```

Q2. Generate and Score a Hypothesis

```
def score_query(query, df):
    """Return all matching entries ranked by confidence (keyword overlap score)."""
    query_words = set(query.lower().split())
    results = []
    for idx, row in df.iterrows():
        kw_words = set(row["keywords"].lower().split())
        score = len(query_words & kw_words)
        if score > 0:
            results.append({
                "index": idx,
                "question": row["question"],
                "answer": row["answer"],
                "category": row["category"],
                "score": score
            })
    results.sort(key=lambda r: r["score"], reverse=True)
    return results

result = score_query("how can I pay the fee", faq_df)
print(pd.DataFrame(result))
```

Output:

```
   index               question category  score
0      3   how can i pay the fee  billing      2
1      0  what is the annual fee  billing      1
```

Q3. same_category(category_name, df)

```
def same_category(category_name, df):
    return df[df["category"] == category_name]

# Called using the category of a personalized entry from Q1 ("billing")
result = same_category("billing", faq_df)
print(result[["question", "category"]])
```

Output:

```
                              question category
0                 what is the annual fee  billing
3                  how can i pay the fee  billing
4  how do i get an invoice for my payment  billing
5        can i pay my fee in installments  billing
```

Q4. Add a New Keyword and Save to CSV

```
new_keyword = input("Enter a new keyword to add: ")   # user input, e.g. "renewal"
entry_to_update = 0   # "what is the annual fee" entry

old_keywords = faq_df.loc[entry_to_update, "keywords"]
faq_df.loc[entry_to_update, "keywords"] = old_keywords + " " + new_keyword

faq_df.to_csv("1024170236_faq_data.csv", index=False)
print("Saved as 1024170236_faq_data.csv")
```

Output (demonstration, entered keyword = 'renewal'):

```
Old keywords: 'fee cost price charge'
New keywords: 'fee cost price charge renewal'

Saved as 1024170236_faq_data.csv
```

Q5. FAQ Entries per Category (groupby)

```
category_counts = faq_df.groupby("category").size()
print(category_counts)
```

Output:

```
category
account    1
billing    4
general    1
dtype: int64
```

Q6. Handling Ties in the Scoring Function

```
def score_query_with_ties(query, df):
    """Score query; if multiple entries tie for the highest score,
    print/return ALL of them instead of silently picking one."""
    query_words = set(query.lower().split())
    results = []
    for idx, row in df.iterrows():
        kw_words = set(row["keywords"].lower().split())
        score = len(query_words & kw_words)
        if score > 0:
            results.append({"index": idx, "question": row["question"],
                            "answer": row["answer"], "category": row["category"],
                            "score": score})
    if not results:
        print("No matches found.")
        return []

    max_score = max(r["score"] for r in results)
    top_matches = [r for r in results if r["score"] == max_score]

    if len(top_matches) > 1:
        print(f"TIE DETECTED - {len(top_matches)} entries tied with score {max_score}:")
        for r in top_matches:
            q, c, s = r["question"], r["category"], r["score"]
            print(f"  - {q}  (category: {c}, score: {s})")
    else:
        best_q = top_matches[0]["question"]
        print(f"Best match (score {max_score}): {best_q}")

    return top_matches
```

Demonstration 1 - a query that produces a TIE:

```
score_query_with_ties("fee", faq_df)
# "fee" appears in BOTH billing entries -> both score 1 -> tie
TIE DETECTED - 2 entries tied with score 1:
  - what is the annual fee  (category: billing, score: 1)
  - how can i pay the fee  (category: billing, score: 1)
```

Demonstration 2 - a query with a single clear winner:

```
score_query_with_ties("password reset login", faq_df)
Best match (score 3): how to reset password
```